

Sarjy — PRD / TDD

Author: Own Abu Hamour **Date:** 10 Aug 2026 **Status:** Draft — shared before implementation, per the take-home rubric **Repo:** sarjy · draft PR opened day 0

1. What I'm building

A deployed, browser-accessible **voice assistant** that listens, answers, remembers you between sessions, and takes real actions through external APIs — in **Arabic and English, including mid-sentence code-switching**.

Framed as a personal assistant (matching the brief's own examples), but the headline capability is a multi-step scheduling flow, because that's where voice assistants actually earn their keep.

The demo, in three lines

Utterance	What it exercises
"سرجي، شو لوني المفضل؟"	Cross-session memory recall, Arabic
"What's on my plate tonight?"	Memory + calendar, English
"احجزلي اجتماع بعد المغرب بكرة"	Prayer-time API → calendar availability → create event → confirm in Arabic

That third one is the thesis of the whole project: it's a multi-step tool chain, it only makes sense in Arabic, and it encodes a piece of local product knowledge that a generic assistant doesn't have.

Non-goals

Stated up front so scope stays honest:

- **No telephony.** LiveKit SIP would make this a phone agent for roughly half a day, and I'll cover the design in the presentation — but I'd rather spend that half-day making the web experience genuinely good than have two mediocre channels.
 - **No video avatar.** High visual cost, low engineering signal.
 - **No multimodal image input.** Orthogonal to the voice problems I find interesting.
 - **Not a production auth system.** Anonymous per-browser identity + a "forget me" button.
-

2. What I'm optimising for

The brief offers ~10 issues to explore and says to pick a couple. I picked three, plus three things that aren't on the list.

Committed — from the brief

Latency. The number a voice team judges instantly. I'm instrumenting every stage of the turn and putting the waterfall on screen, live.

Language switching (AR/EN). Not just "supports Arabic" — mid-sentence code-switching, and an honest measured comparison of what Arabic costs versus English across ASR accuracy, endpointing, and latency.

Cost at 100 / 1k / 10k users. Measured per-minute unit economics, not a guess.

Quality/fidelity rides along: noise cancellation and barge-in are near-free on this stack, and "does turn detection actually work in Arabic?" is a question that belongs to the language pillar anyway.

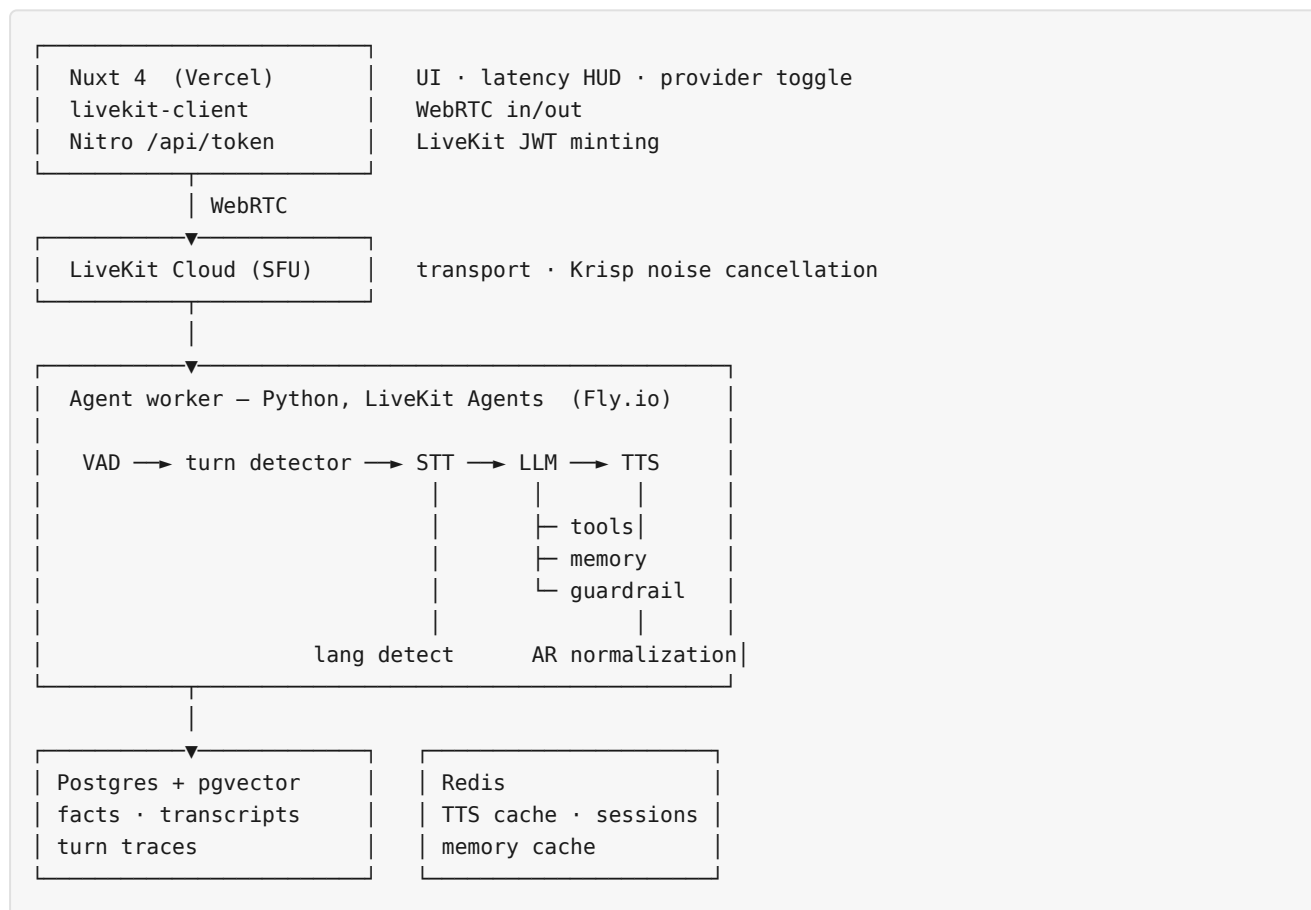
Committed — not in the brief

Arabic TTS text normalization. Arabic TTS mangles exactly what an assistant says constantly: numbers, times, dates, currency, and Latin brand names embedded in Arabic. A normalization pass before synthesis, presented as a before/after A/B.

Graceful degradation. Voice has no spinner — silence *is* the failure. Provider failover, cached filler audio while retrying, and a recovery path when ASR returns garbage.

A small eval harness in CI. ~12 recorded audio fixtures (clean AR, clean EN, code-switched, noisy) producing a scorecard on every push: WER per language, turn latency p50/p95, tool-call accuracy. Small on purpose. The point is that regressions are caught by a machine, not by me listening.

3. Architecture



Stack and why

Layer	Choice	Reasoning
Web	Nuxt 4 + livekit-client	LiveKit ships React components but no Vue package, so I use the core SDK via composables. The HUD is custom code either way, so I lose nothing. Nitro gives me a clean token endpoint.

Layer	Choice	Reasoning
Transport	LiveKit Cloud	WebRTC, jitter buffering, Krisp noise cancellation, and a SIP path if telephony ever matters. Building this by hand would consume the whole budget.
Agent	LiveKit Agents (Python)	The mature side of the ecosystem — better plugin coverage and the multilingual turn detector. The worker is small; the language is a means, not a statement.
STT	Groq Whisper large-v3-turbo , ElevenLabs Scribe as comparator	Groq is the fastest hosted Whisper. Scribe is stronger on Arabic; I'll measure both rather than assert.
LLM	Groq (latency) / Gemini Flash (Arabic quality)	Selected per request behind one interface, so provider choice becomes a measured slide instead of an opinion.
TTS	Adapter — Gemini default , ElevenLabs Flash v2.5 if a key is available	See open question §8. Provider-agnostic by design, which also produces a real comparison.
Memory	Postgres + pgvector	Extracted facts, embedded and retrieved. Not raw history stuffed into a prompt.
Cache	Redis	TTS phrases, memory retrieval, semantic answers.

Why cascaded, not speech-to-speech

Realtime speech-to-speech models (Gemini Live, OpenAI Realtime) are meaningfully faster and more natural. I'm not using one, deliberately:

- I can't instrument inside a black box, and per-stage latency is the point of this submission.
- Per-language provider swapping is impossible.
- Memory injection and guardrails become prompt-only.
- Cost per minute is several times higher, which undermines the scaling analysis.

A cascade is the slower, more controllable choice, and control is what I'm demonstrating. I'll include a measured comparison against Gemini Live so the tradeoff is quantified rather than asserted.

4. Latency

The metric is **end of user speech** → **first audible byte of response**.

The table below is a **pre-build target, not a measurement** — nothing has been benchmarked yet, since the agent doesn't exist yet. Figures are derived from each named provider's published latency numbers (Groq's Whisper/LLM inference speed, ElevenLabs Flash v2.5's stated TTFB, LiveKit's turn-detector docs), used here to set a budget I'll build against. **Real p50/p95 numbers, measured on this system, are a Tier 1 deliverable** — reported live via the on-screen HUD (§3) and tracked per-commit by the CI eval harness (§2) — and I expect the real numbers to disagree with this table in places, Arabic especially (see below).

Stage	Target (EN)	Basis
Endpointing / turn detection	300–500 ms	LiveKit semantic turn detector, published range. Expected dominant cost.

Stage	Target (EN)	Basis
STT finalisation	150–300 ms	Groq's published Whisper large-v3-turbo inference speed; streaming, so most audio is already transcribed.
Memory retrieval	< 20 ms	Design target — Redis-cached, must not be on the critical path.
LLM first token	150–250 ms	Groq's published TTFT; assumes prompt caching on system + memory block.
TTS first byte	150–400 ms	ElevenLabs Flash v2.5 vs. Gemini TTS published figures — widest variance, most provider-dependent.
Transport	50–100 ms	Typical WebRTC RTT for a well-placed SFU. Region-dependent, see below.

Targets: EN p50 ≤ 900 ms · AR p50 ≤ 1200 ms.

I expect to miss the Arabic target somewhere. Endpointing models are trained overwhelmingly on English, Arabic speakers pause differently, and code-switching can look like an endpoint. **If I miss it, I'll report the number I got and explain why** — that's more useful to a voice team than a target quietly moved.

Caching

Five layers, each with a specific job:

1. **TTS phrase cache** — pre-rendered fixed strings (greetings, "لحظة من فضلك", confirmations, tool-call fillers) keyed by (text, voice, lang). Takes first-byte to ~0 ms and cuts the largest per-minute cost line.
2. **Warm provider connections** — STT/TTS sockets opened at session start, not per turn. Removes a TLS handshake from every single turn.
3. **LLM prompt caching** — system prompt and memory block, for TTFT and cost.
4. **Memory retrieval cache** — Redis in front of pgvector, keeping a DB round-trip off the critical path.
5. **Semantic answer cache** — embedding-keyed, for repeated questions. Matters mainly at scale; feeds \$6.

Region

Users are in KSA. Where the model providers actually serve from is less settled than it first looks. Groq operates a large inference facility in Dammam with Aramco Digital (reportedly EMEA's biggest AI inference deployment), built for Saudi sovereign AI workloads — but there's no public confirmation that a standard `api.groq.com` developer key routes there rather than to US infrastructure; that capacity may be earmarked for enterprise/sovereign contracts. Google Cloud has a Dammam region too (me-central2), but it's gated behind their KSA reseller (CNTXT) — the consumer Gemini API key is a different product and almost certainly doesn't route through it.

So rather than assume either way, I'm placing the LiveKit room close to the user (transport, where RTT hurts most on every packet) and **testing which region I'm actually hitting once the agent is running** — a TTFB/traceroute comparison against a known-US-only endpoint will show it. If it turns out Groq is serving from Dammam, that's a meaningfully tighter latency budget than the table above assumes, and worth knowing either way.

5. Memory

Three tiers, because "remembers conversations" means different things at different timescales:

Tier	Store	Lifetime	Use
Turn context	In-process	Current call	Last N turns verbatim
Session summary	Postgres	Per conversation	Rolling summary, keeps the prompt bounded
Long-term facts	Postgres + pgvector	Permanent	favorite_color = أزرق, extracted and embedded

Facts are **extracted, not accumulated** — after each turn, a cheap model pulls durable statements ("my favourite colour is blue") and discards transient ones ("what's the weather"). Retrieval is a vector search scoped to the user, injected as a compact block rather than a transcript dump.

Two details I care about:

Barge-in truncation. When the user interrupts, history must be truncated to *what was actually spoken aloud*, not the full generated response. Get this wrong and memory silently records things the user never heard — a bug that only surfaces days later, as a wrong answer with no visible cause.

Memory feeding ASR. Known entities — the user's name, contacts, city — are passed to STT as vocabulary hints, so Arabic ASR stops mangling proper nouns. Memory improving recognition, not just recall.

6. Cost

Measured per-minute unit cost, decomposed by stage, extrapolated to 100 / 1k / 10k users at an assumed conversation profile (stated explicitly in the model, not hidden).

```
cost/min = STT(audio_min) + LLM(tokens_in, tokens_out) + TTS(chars) + transport(min) + infra
```

The analysis I actually want to present: **which line dominates, and what each cache layer removes from it.** TTS is usually the biggest line and is also the most cacheable — that's the interesting result, and it connects the cost slide back to the latency work rather than leaving it as a standalone spreadsheet.

7. Plan

Tiered, so that finishing early or running late is a scope decision rather than a scramble.

Tier 0 — Core (must ship). Voice loop in AR + EN, deployed and publicly reachable, cross-session memory, prayer-time + calendar tool chain, README.

Tier 1 — Committed (the actual submission). Latency instrumentation + live HUD, caching layers, Arabic/English measured comparison, Arabic TTS normalization, graceful degradation, cost model, eval harness.

Tier 2 — Stretch (only if ahead). Guardrails as a fast parallel classifier with its own latency budget. Speech-to-speech comparison against Gemini Live. Richer eval fixtures. Telephony spike.

Day	Target
0	This document. Repo, draft PR, API key request.
1	End-to-end voice loop, both languages, deployed same day . Basic memory.

Day	Target
2	pgvector memory, tool chain, latency instrumentation + HUD, cache layers.
3	Arabic normalization, degradation paths, eval harness, cost model.
4 (<i>buffer</i>)	Polish, presentation, or Tier 2 if days 1–3 went well.

Deploying on day 1 rather than day 3 is deliberate — deployment surprises are the classic way this kind of project fails at the last hour.

Daily updates regardless of progress, including the days where the update is "no time today".

8. Open questions and asks

- ElevenLabs key?** It's the strongest Arabic TTS and Flash v2.5 is the lowest-latency option, but it's not on the Groq/Gemini/SambaNova list. I'm building TTS behind an adapter either way and defaulting to Gemini — a key would meaningfully improve the "natural Saudi voice" side of the demo. **Groq and Gemini keys requested now.**
 - Dialect target?** I'm defaulting to Levantine/MSA-neutral input handling with MSA output. If Saudi dialect specifically matters for how you'd assess this, that changes my STT and voice selection.
 - Regional routing** — Groq operates a large inference facility in Dammam with Aramco Digital; I can't confirm from public docs whether a standard developer API key routes there or to US infrastructure. If you already know the answer, it'd save me a measurement — otherwise I'll test it directly once the agent is running (\$4).
 - Presentation format** — I'll send a Loom plus this document before the session, so the 5 minutes can be questions rather than narration.
-

9. Risks

Risk	Mitigation
Arabic endpointing underperforms	Expected. Measuring and reporting it is a deliverable, not a failure.
TTS quality gates the demo	Provider adapter + normalization layer; the demo survives a weaker provider.
Scope creep across ~10 explore items	Tiered plan above. Three committed pillars, everything else is Tier 2.
Deploy problems at the end	Deployed day 1, redeployed continuously.
Free-tier rate limits during the live demo	Cached responses for the scripted path + a recorded fallback video.